ORDER MANAGEMENT SYSTEM

1. Introduction

The Order Management System stores customer orders, the products included in each order, quantities, prices, and order status. It supports order creation, updates, cancellation, detailed order viewing, and customer/product sales analysis.

2. Objectives

Create relational tables for customers, products, orders, and order details.

Apply primary keys, foreign keys, NOT NULL, UNIQUE, and CHECK constraints.

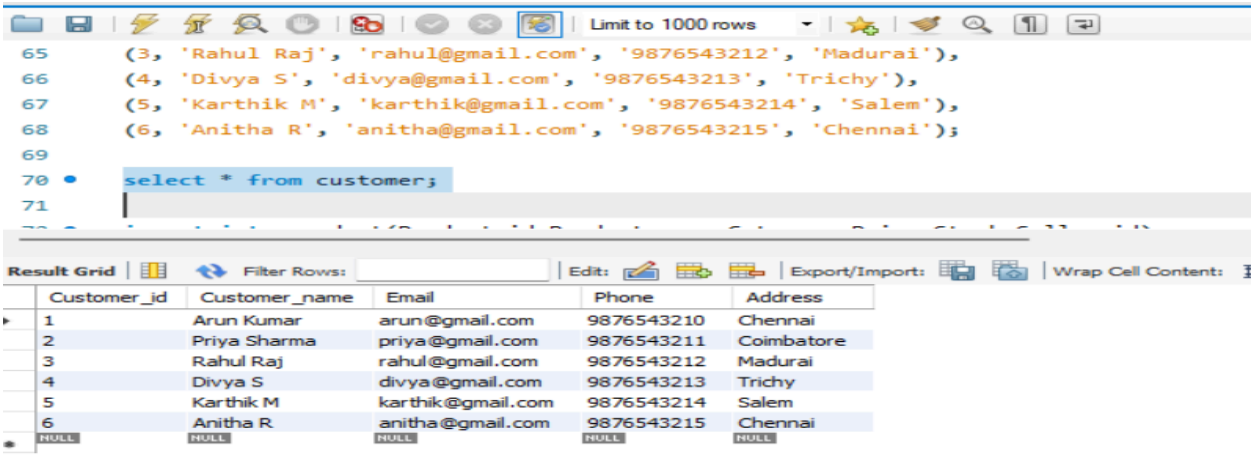
Record multiple products in one order and calculate order totals.

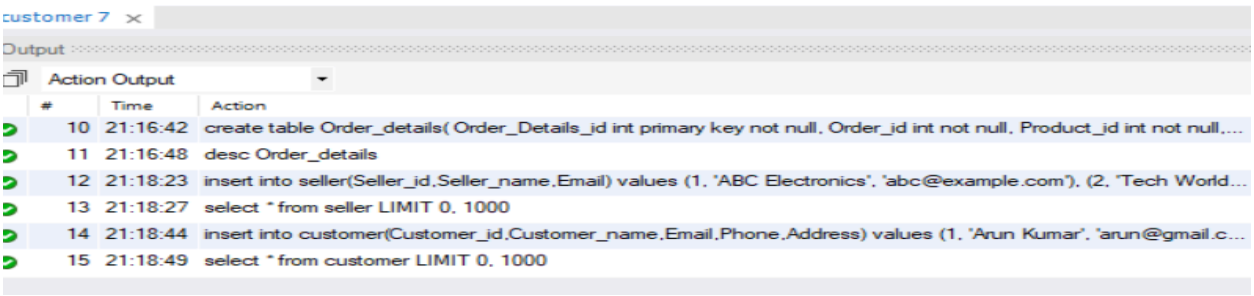
Track Pending, Shipped, Delivered, and Cancelled statuses.

Generate customer order history and product/sales reports.

3. Table Design

1) Customer





Customer_ID INT PRIMARY KEY

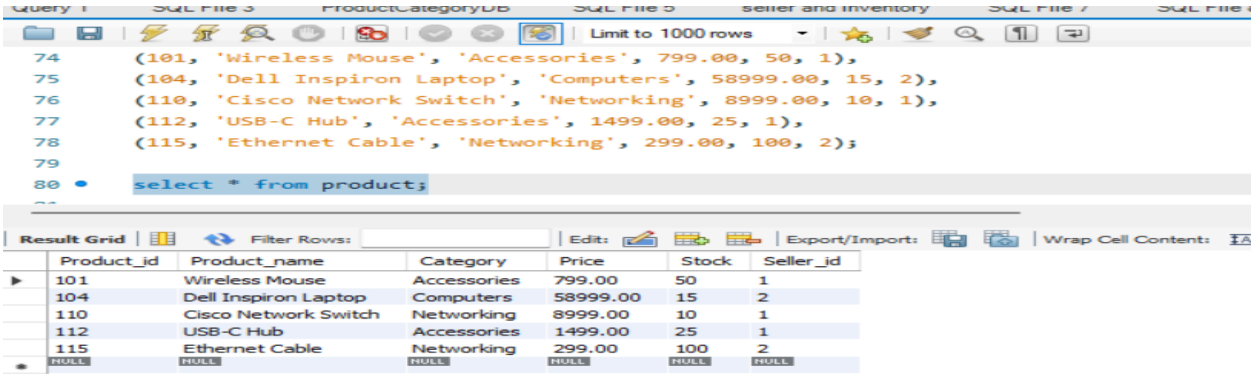
Customer_Name VARCHAR(100) NOT NULL

Email VARCHAR(100) UNIQUE NOT NULL

Phone VARCHAR(15) UNIQUE NOT NULL

Address VARCHAR(200) NOT NULL

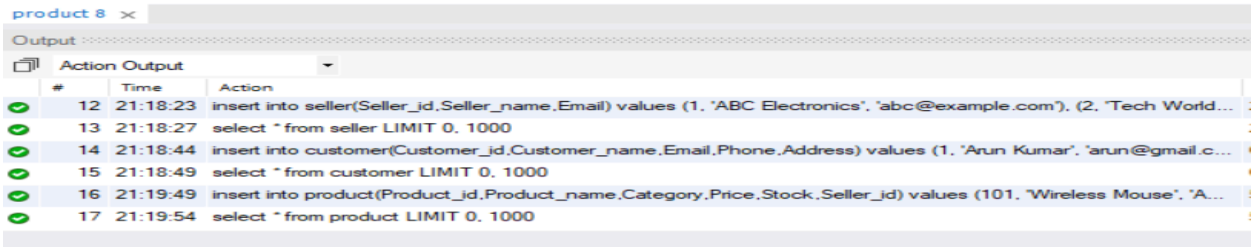
2) Product



```
74      (101, 'Wireless Mouse', 'Accessories', 799.00, 50, 1),
75      (104, 'Dell Inspiron Laptop', 'Computers', 58999.00, 15, 2),
76      (110, 'Cisco Network Switch', 'Networking', 8999.00, 10, 1),
77      (112, 'USB-C Hub', 'Accessories', 1499.00, 25, 1),
78      (115, 'Ethernet Cable', 'Networking', 299.00, 100, 2);
79
80      select * from product;
```

Result Grid

Product_id	Product_name	Category	Price	Stock	Seller_id
101	Wireless Mouse	Accessories	799.00	50	1
104	Dell Inspiron Laptop	Computers	58999.00	15	2
110	Cisco Network Switch	Networking	8999.00	10	1
112	USB-C Hub	Accessories	1499.00	25	1
115	Ethernet Cable	Networking	299.00	100	2



product 8

Output

Action Output

#	Time	Action
12	21:18:23	insert into seller(Seller_id,Seller_name,Email) values (1, 'ABC Electronics', 'abc@example.com'), (2, 'Tech World...
13	21:18:27	select * from seller LIMIT 0, 1000
14	21:18:44	insert into customer(Customer_id,Customer_name,Email,Phone,Address) values (1, 'Arun Kumar', 'arun@gmail.c...
15	21:18:49	select * from customer LIMIT 0, 1000
16	21:19:49	insert into product(Product_id,Product_name,Category,Price,Stock,Seller_id) values (101, 'Wireless Mouse', 'A...
17	21:19:54	select * from product LIMIT 0, 1000

Product_ID INT PRIMARY KEY

Product_Name VARCHAR(100) NOT NULL

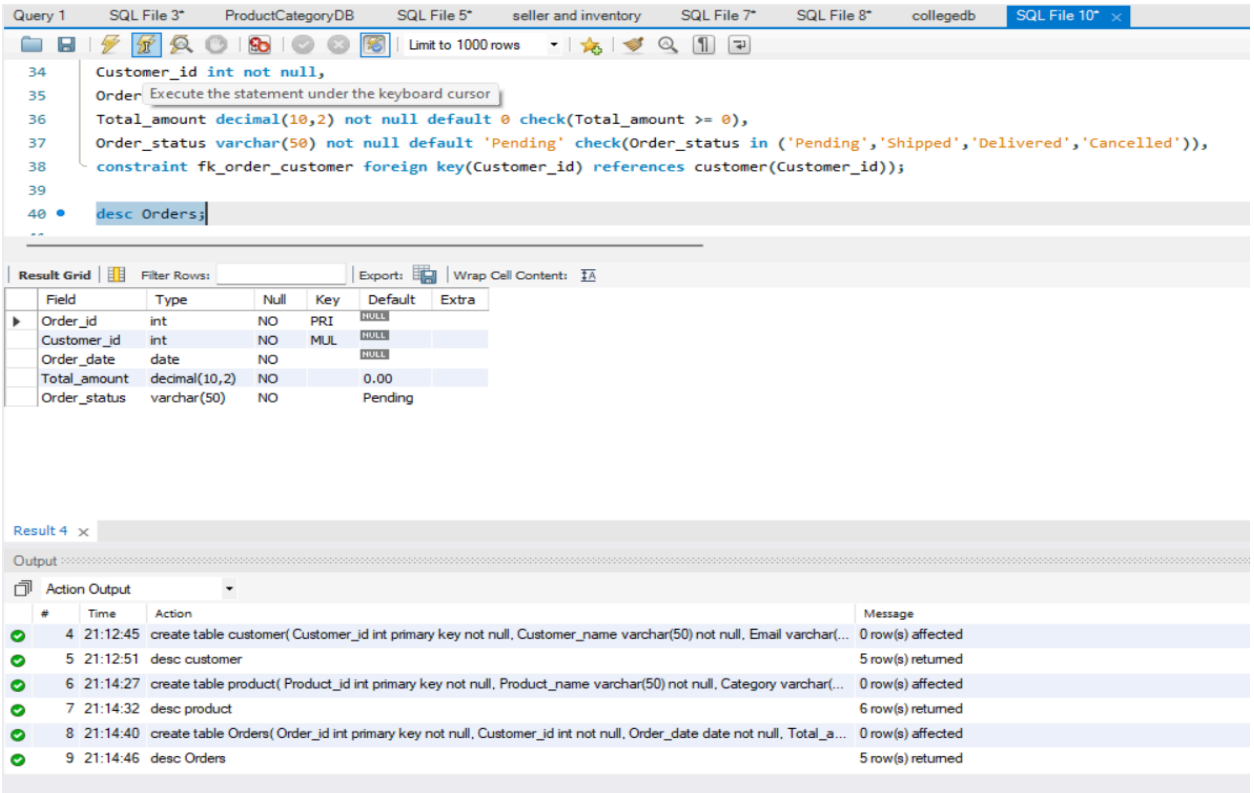
Category VARCHAR(50) NOT NULL

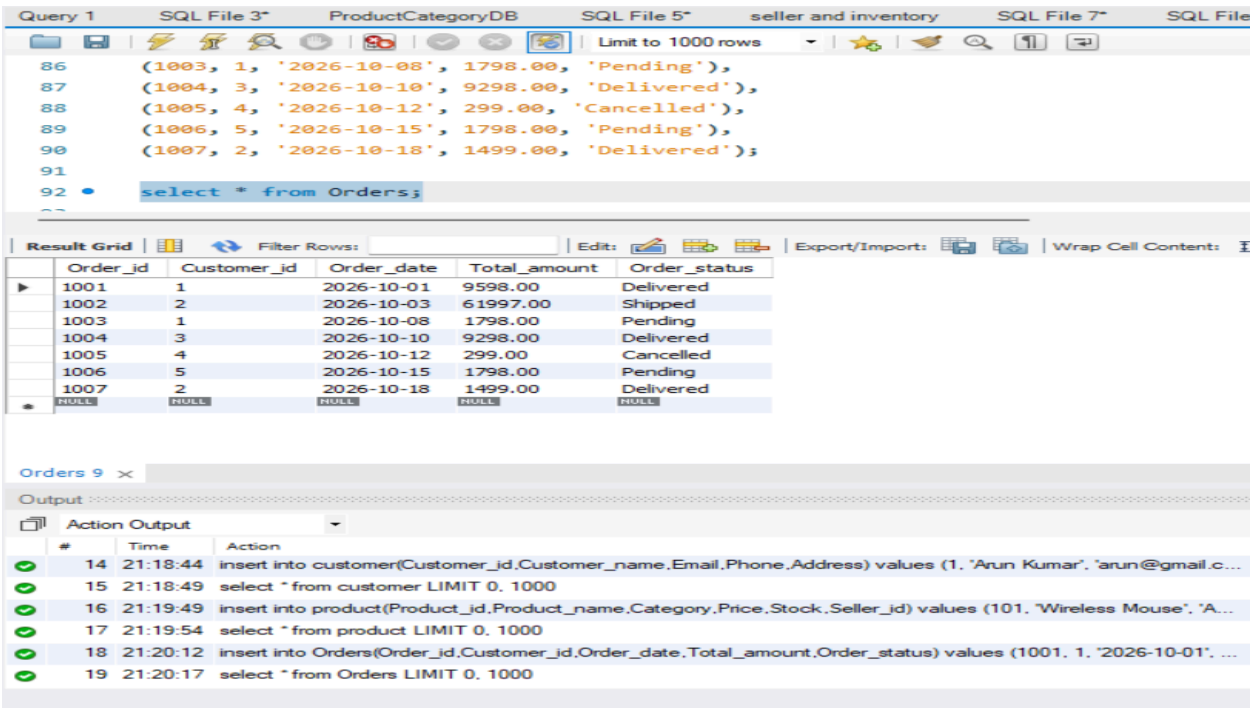
Price DECIMAL(10,2) CHECK (Price > 0)

Stock INT CHECK (Stock >= 0)

Seller_ID INT FOREIGN KEY

3) Order


Query 1
SQL File 3*
ProductCategoryDB
SQL File 5*
seller and inventory
SQL File 7*
SQL File 8*
collegedb
SQL File 10*
Limit to 1000 rows
Customer_id int not null,
Order
Execute the statement under the keyboard cursor
Total_amount decimal(10,2) not null default 0 check(Total_amount >= 0),
Order_status varchar(50) not null default 'Pending' check(Order_status in ('Pending','Shipped','Delivered','Cancelled')),
constraint fk_order_customer foreign key(Customer_id) references customer(Customer_id));
desc Orders;
Result Grid
Filter Rows:
Export:
Wrap Cell Content:
Field Type Null Key Default Extra
Order_id int NO PRI NULL
Customer_id int NO MUL NULL
Order_date date NO NULL
Total_amount decimal(10,2) NO 0.00
Order_status varchar(50) NO Pending
Result 4
Output
Action Output
Time Action Message
4 21:12:45 create table customer(Customer_id int primary key not null, Customer_name varchar(50) not null, Email varchar(... 0 row(s) affected
5 21:12:51 desc customer 5 row(s) returned
6 21:14:27 create table product(Product_id int primary key not null, Product_name varchar(50) not null, Category varchar(... 0 row(s) affected
7 21:14:32 desc product 6 row(s) returned
8 21:14:40 create table Orders(Order_id int primary key not null, Customer_id int not null, Order_date date not null, Total_a... 0 row(s) affected
9 21:14:46 desc Orders 5 row(s) returned


Query 1
SQL File 3*
ProductCategoryDB
SQL File 5*
seller and inventory
SQL File 7*
SQL File
Limit to 1000 rows
(1003, 1, '2026-10-08', 1798.00, 'Pending'),
(1004, 3, '2026-10-10', 9298.00, 'Delivered'),
(1005, 4, '2026-10-12', 299.00, 'Cancelled'),
(1006, 5, '2026-10-15', 1798.00, 'Pending'),
(1007, 2, '2026-10-18', 1499.00, 'Delivered');
select * from Orders;
Result Grid
Filter Rows:
Edit:
Export/Import:
Wrap Cell Content:
Order_id Customer_id Order_date Total_amount Order_status
1001 1 2026-10-01 9598.00 Delivered
1002 2 2026-10-03 61997.00 Shipped
1003 1 2026-10-08 1798.00 Pending
1004 3 2026-10-10 9298.00 Delivered
1005 4 2026-10-12 299.00 Cancelled
1006 5 2026-10-15 1798.00 Pending
1007 2 2026-10-18 1499.00 Delivered
Orders 9
Output
Action Output
Time Action
14 21:18:44 insert into customer(Customer_id,Customer_name,Email,Phone,Address) values (1, 'Arun Kumar', 'arun@gmail.c...
15 21:18:49 select * from customer LIMIT 0, 1000
16 21:19:49 insert into product(Product_id,Product_name,Category,Price,Stock,Seller_id) values (101, 'Wireless Mouse', 'A...
17 21:19:54 select * from product LIMIT 0, 1000
18 21:20:12 insert into Orders(Order_id,Customer_id,Order_date,Total_amount,Order_status) values (1001, 1, '2026-10-01', ...
19 21:20:17 select * from Orders LIMIT 0, 1000

Order_ID INT PRIMARY KEY

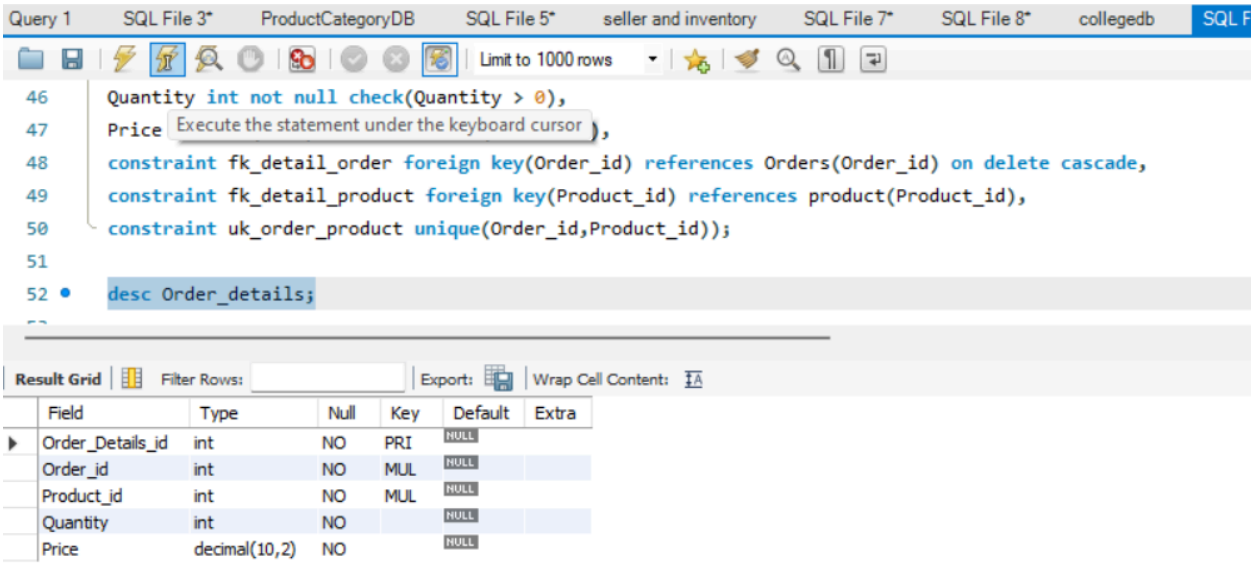
Customer_ID INT NOT NULL FOREIGN KEY → Customer

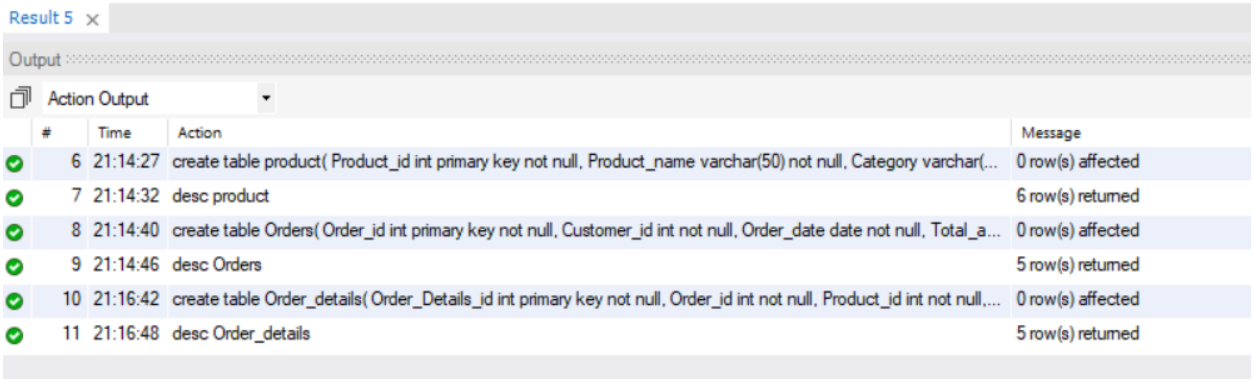
Order_Date DATE NOT NULL

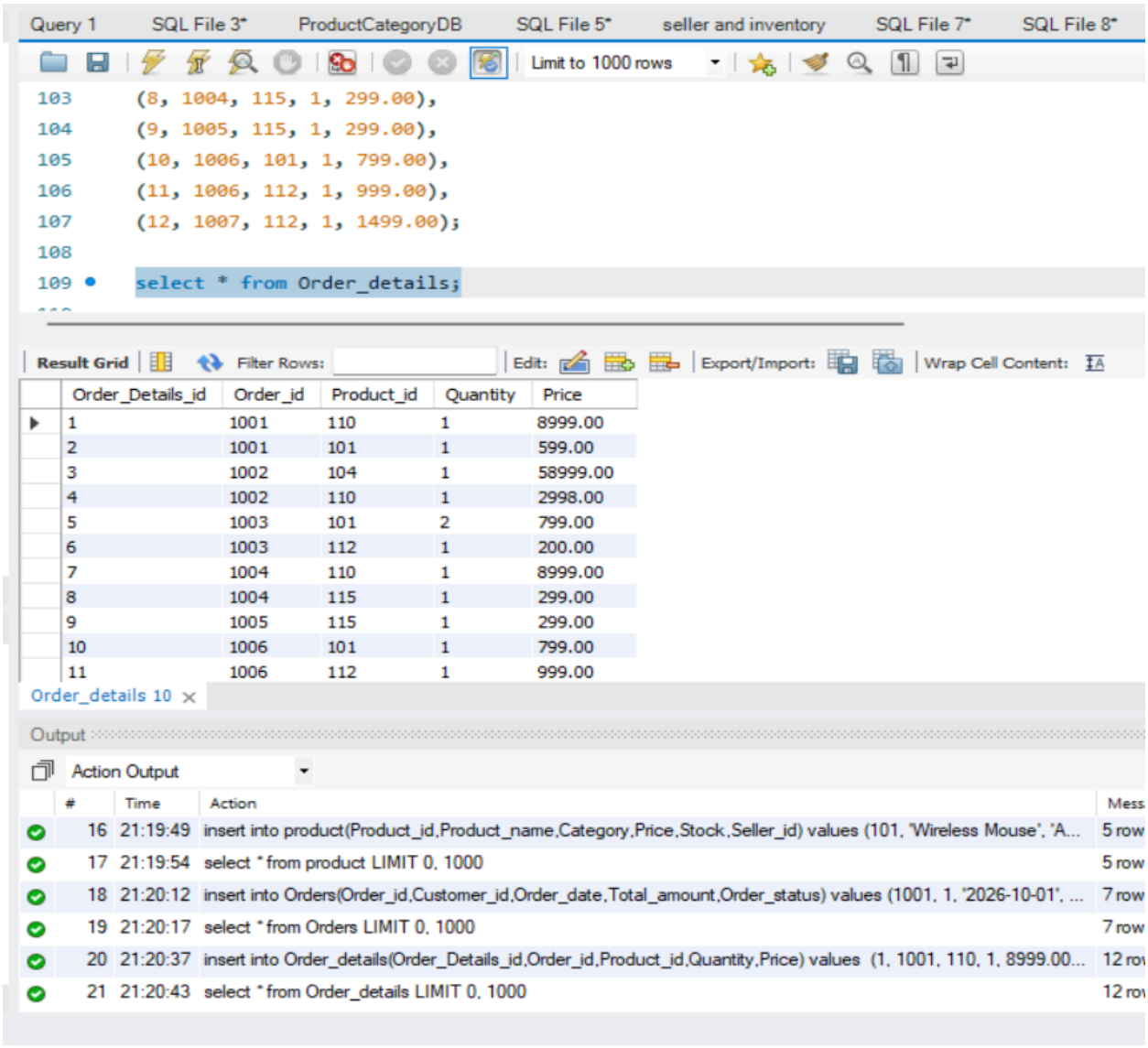
Total_Amount DECIMAL(10,2) DEFAULT 0 CHECK (Total_Amount >= 0)

Order_Status VARCHAR(20) DEFAULT 'Pending' CHECK (status is allowed)

4) Order Details







Order_Detail_ID INT PRIMARY KEY

Order_ID INT NOT NULL FOREIGN KEY → Orders

Product_ID INT NOT NULL FOREIGN KEY → Product

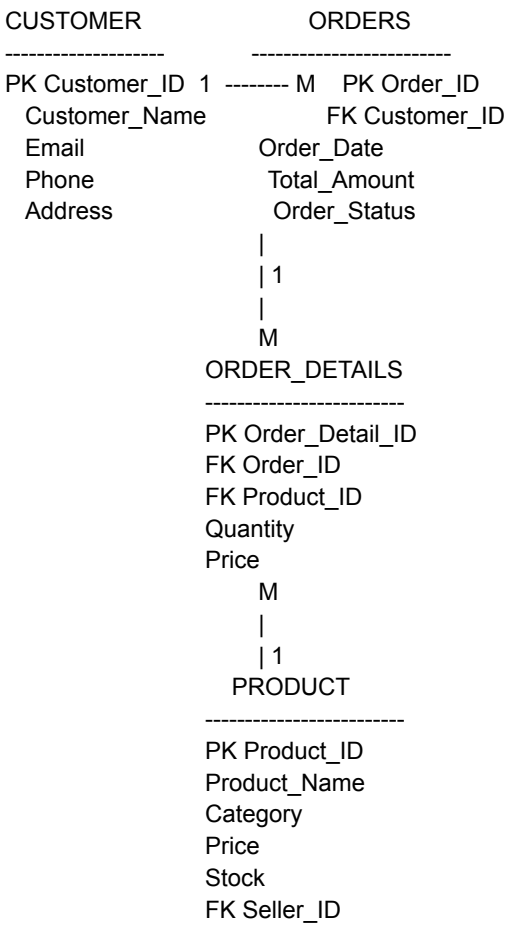
Quantity INT NOT NULL CHECK (Quantity > 0)

Price DECIMAL(10,2) NOT NULL CHECK (Price > 0)

UNIQUE (Order_ID, Product_ID)

4. Relationships and ER Diagram

Customer (1) —— (M) Orders; Orders (1) —— (M) Order_Details; Product (1) —— (M) Order_Details. Order_Details is the associative table that resolves the many-to-many relationship between Orders and Product.



5. Constraints

PRIMARY KEY: Uniquely identifies each row in a table.

FOREIGN KEY: Ensures referenced customer, order, product, and seller records exist.

NOT NULL: Requires important fields such as customer name, order date, quantity, and price.

UNIQUE: Prevents duplicate customer emails/phone numbers and duplicate product lines within one order.

CHECK: Requires positive product prices and quantities, non-negative stock/totals, and valid order statuses.

ON DELETE CASCADE: Deleting an order automatically deletes its associated Order_Details rows.

6. Sample Data

The SQL script inserts sample sellers, six customers, five products, seven orders, and twelve order-detail rows. Product examples are Wireless Mouse (₹799), Dell Inspiron Laptop (₹58,999), Cisco Network Switch (₹8,999), USB-C Hub (₹1,499), and Ethernet Cable (₹299).

Order totals are calculated from the order-detail rows as Quantity × Price. In a real deployment, keep the stored Orders.Total_Amount synchronized whenever details change; the script demonstrates explicit recalculation queries.

7. Order Management Operations

Create a new order with a customer ID and initial Pending status.

Insert multiple product lines into Order_Details.

Calculate the order total using SUM(Quantity × Price).

Update Order_Status to Pending, Shipped, Delivered, or Cancelled.

Change a line's Quantity and recalculate its order total.

Delete cancelled orders; their detail rows are removed through ON DELETE CASCADE.

Join Customer, Orders, Order_Details, and Product to display complete order information.

8. Customer Order History and Purchase Analysis

Customer order history: Lists each customer’s orders, date, status, and total.

Product-wise order report: Shows distinct orders, total quantity, and gross line sales per product.

Times each product was ordered: Counts distinct non-cancelled orders containing each product.

Total quantity sold: Sums ordered quantities, excluding cancelled orders.

Customers with maximum orders: Returns every customer tied for the highest number of non-cancelled orders.

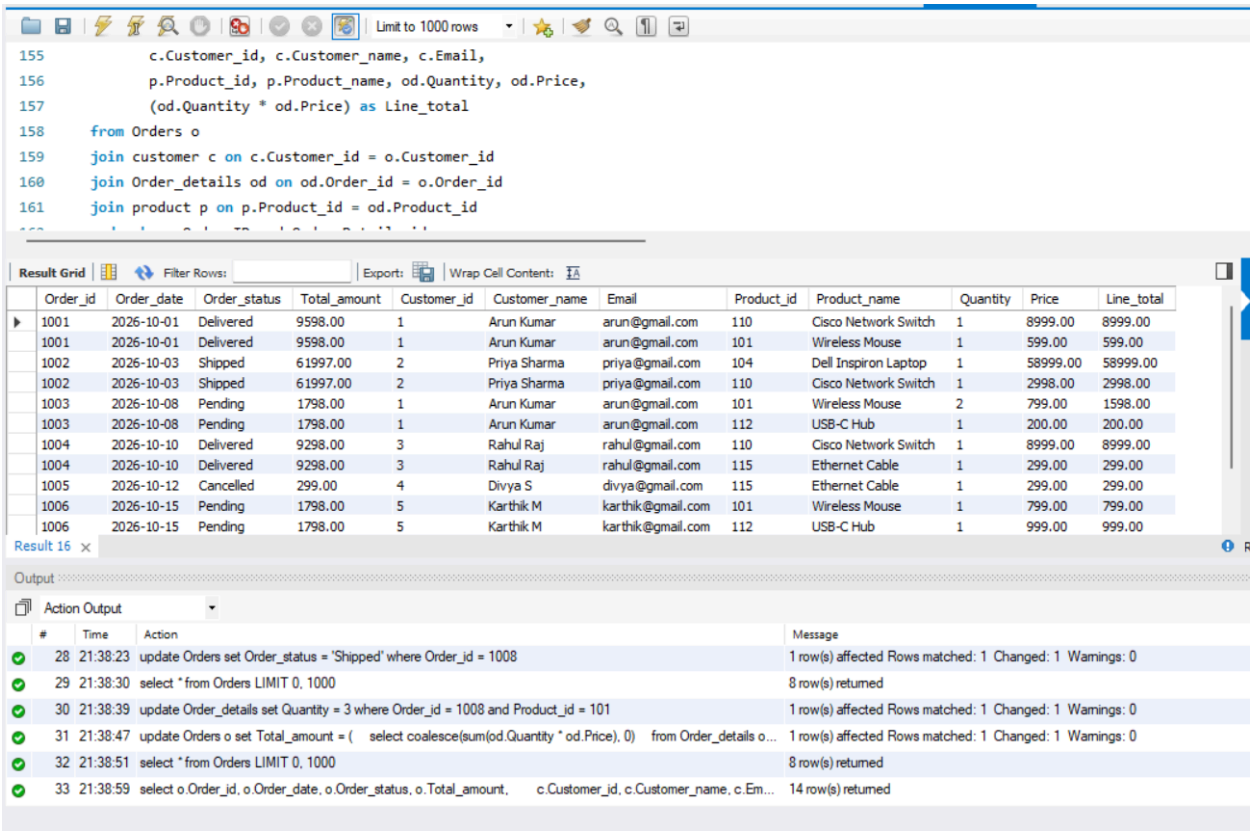
Total spending per customer: Sums non-cancelled order totals per customer.

Average order value: Calculates the average total of non-cancelled orders.

Total sales amount: Sums non-cancelled order totals.

9. Screenshots

1) Order Details Join



```
155          c.Customer_id, c.Customer_name, c.Email,
156          p.Product_id, p.Product_name, od.Quantity, od.Price,
157          (od.Quantity * od.Price) as Line_total
158      from Orders o
159      join customer c on c.Customer_id = o.Customer_id
160      join Order_details od on od.Order_id = o.Order_id
161      join product p on p.Product_id = od.Product_id
```

Result Grid

Order_id	Order_date	Order_status	Total_amount	Customer_id	Customer_name	Email	Product_id	Product_name	Quantity	Price	Line_total
1001	2026-10-01	Delivered	9598.00	1	Arun Kumar	arun@gmail.com	110	Cisco Network Switch	1	8999.00	8999.00
1001	2026-10-01	Delivered	9598.00	1	Arun Kumar	arun@gmail.com	101	Wireless Mouse	1	599.00	599.00
1002	2026-10-03	Shipped	61997.00	2	Priya Sharma	priya@gmail.com	104	Dell Inspiron Laptop	1	58999.00	58999.00
1002	2026-10-03	Shipped	61997.00	2	Priya Sharma	priya@gmail.com	110	Cisco Network Switch	1	2998.00	2998.00
1003	2026-10-08	Pending	1798.00	1	Arun Kumar	arun@gmail.com	101	Wireless Mouse	2	799.00	1598.00
1003	2026-10-08	Pending	1798.00	1	Arun Kumar	arun@gmail.com	112	USB-C Hub	1	200.00	200.00
1004	2026-10-10	Delivered	9298.00	3	Rahul Raj	rahul@gmail.com	110	Cisco Network Switch	1	8999.00	8999.00
1004	2026-10-10	Delivered	9298.00	3	Rahul Raj	rahul@gmail.com	115	Ethernet Cable	1	299.00	299.00
1005	2026-10-12	Cancelled	299.00	4	Divya S	divya@gmail.com	115	Ethernet Cable	1	299.00	299.00
1006	2026-10-15	Pending	1798.00	5	Karthik M	karthik@gmail.com	101	Wireless Mouse	1	799.00	799.00
1006	2026-10-15	Pending	1798.00	5	Karthik M	karthik@gmail.com	112	USB-C Hub	1	999.00	999.00

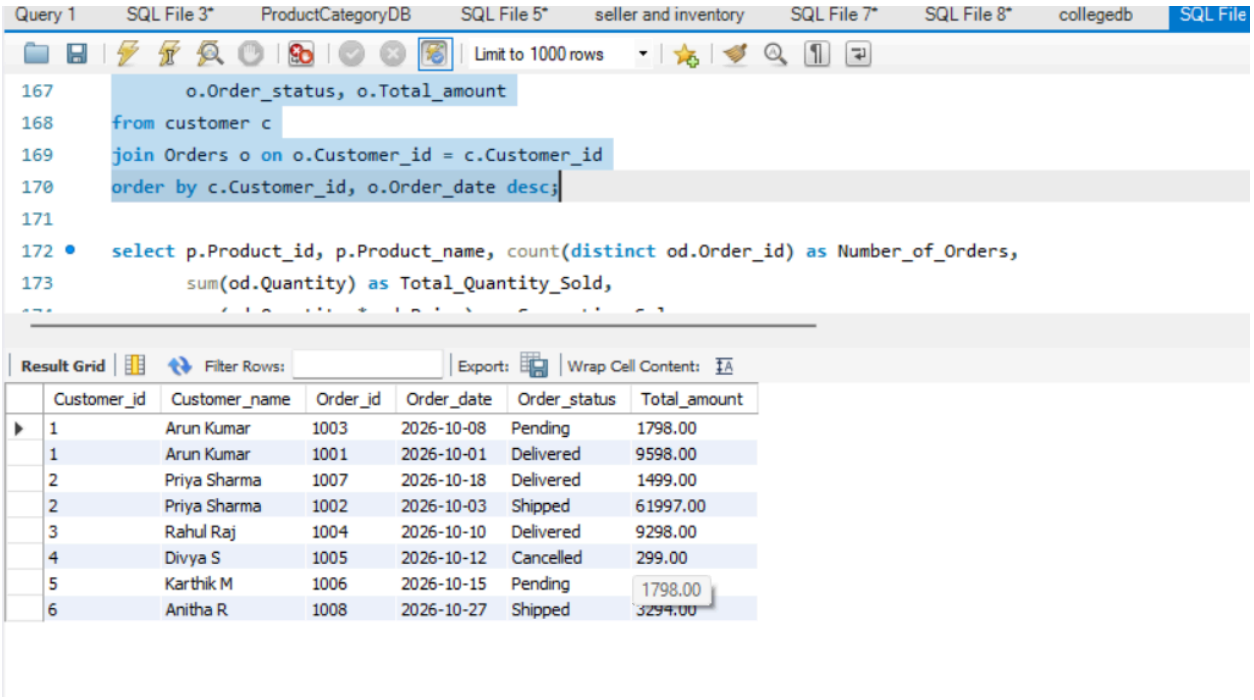
Result 16

Output

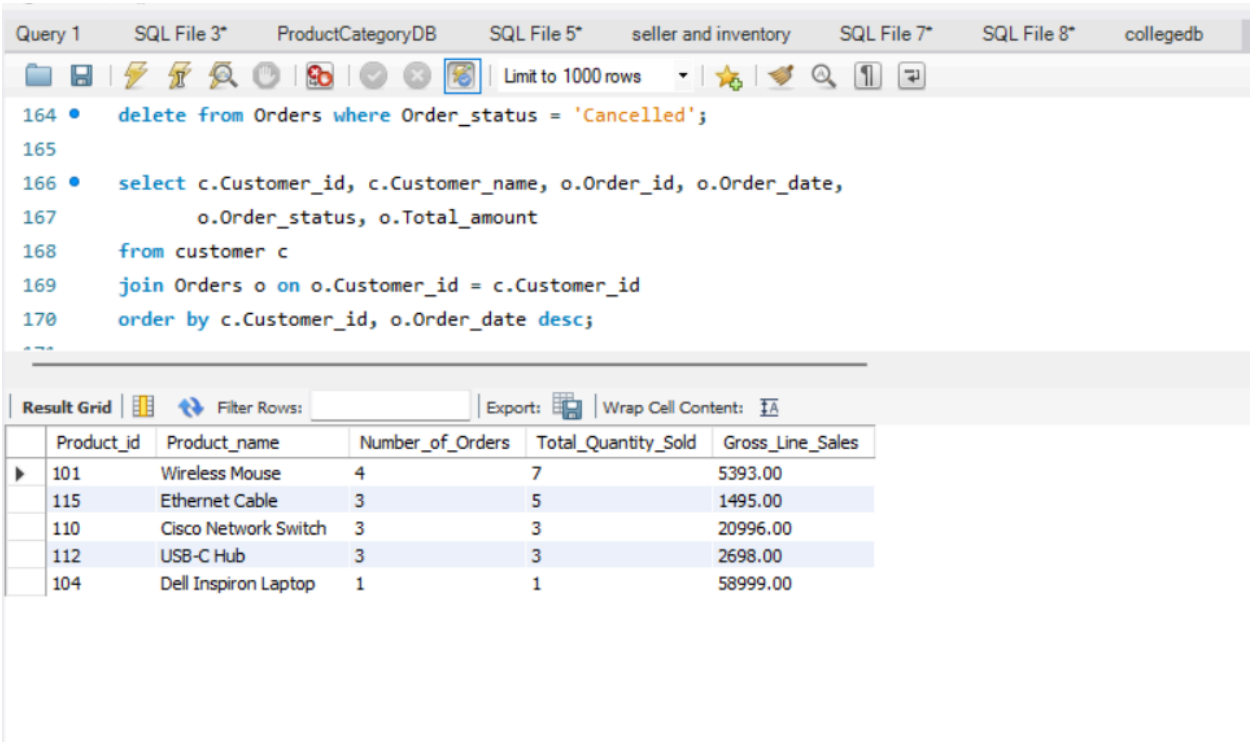
Action Output

#	Time	Action	Message
28	21:38:23	update Orders set Order_status = 'Shipped' where Order_id = 1008	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0
29	21:38:30	select * from Orders LIMIT 0, 1000	8 row(s) returned
30	21:38:39	update Order_details set Quantity = 3 where Order_id = 1008 and Product_id = 101	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0
31	21:38:47	update Orders o set Total_amount = (select coalesce(sum(od.Quantity * od.Price), 0) from Order_details o...	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0
32	21:38:51	select * from Orders LIMIT 0, 1000	8 row(s) returned
33	21:38:59	select o.Order_id, o.Order_date, o.Order_status, o.Total_amount, c.Customer_id, c.Customer_name, c.Em...	14 row(s) returned

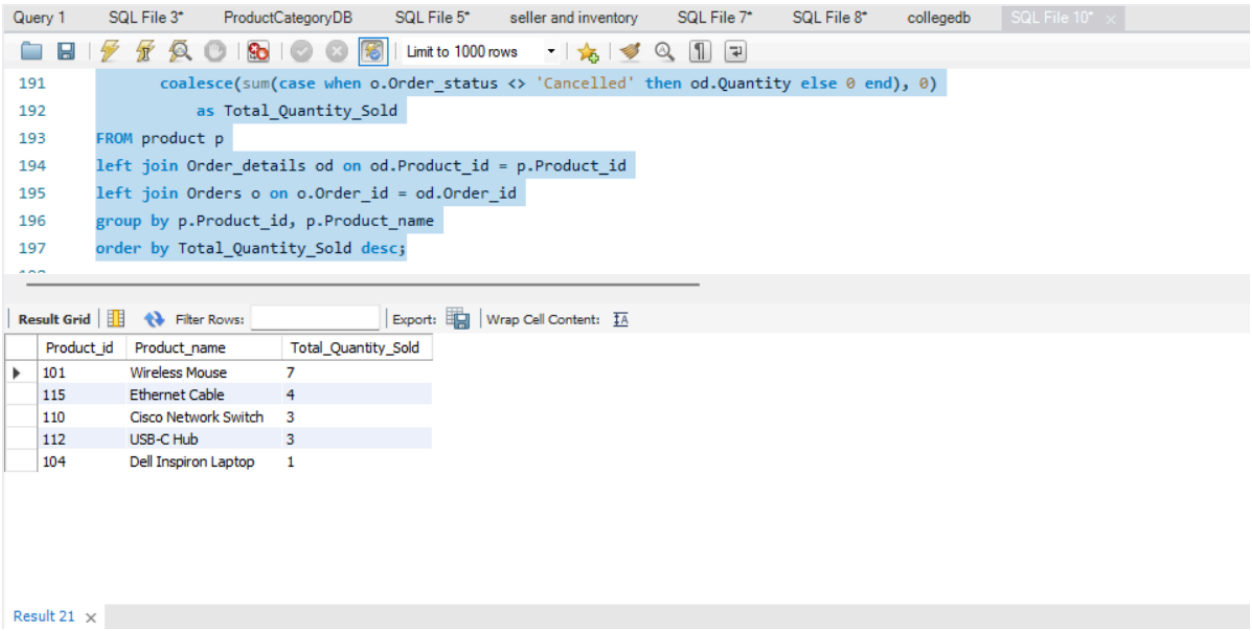
2) Customer Order History



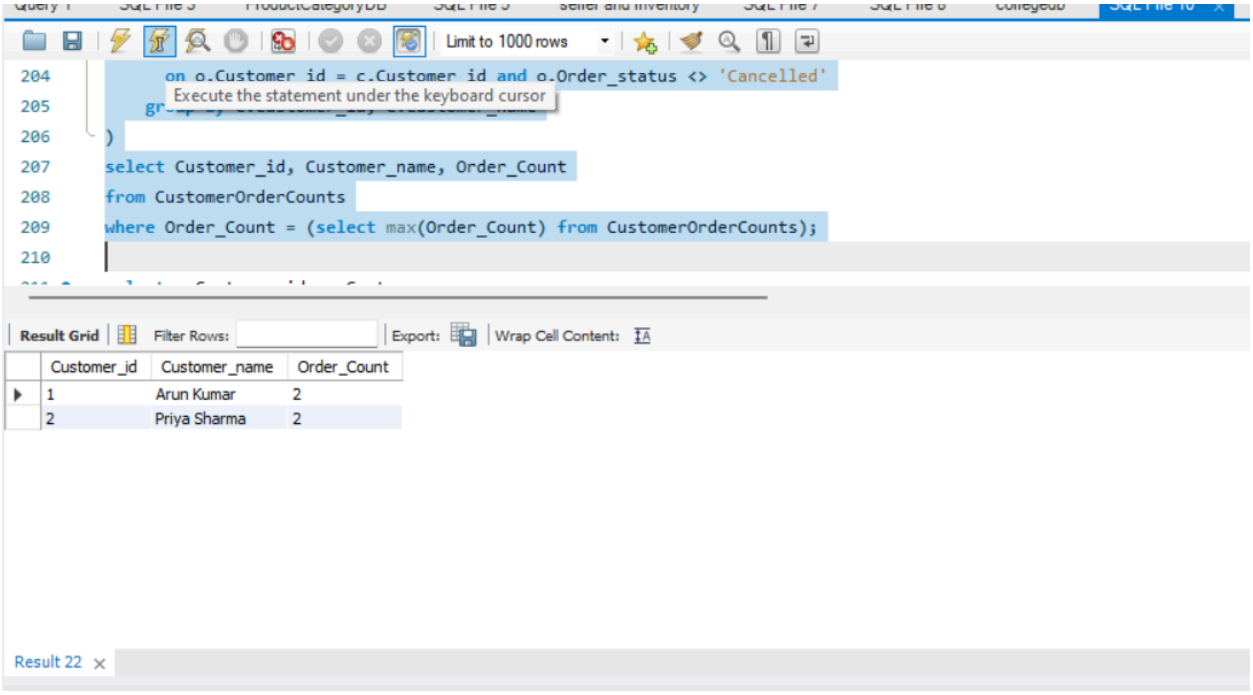
3) Product-Wise Order Report



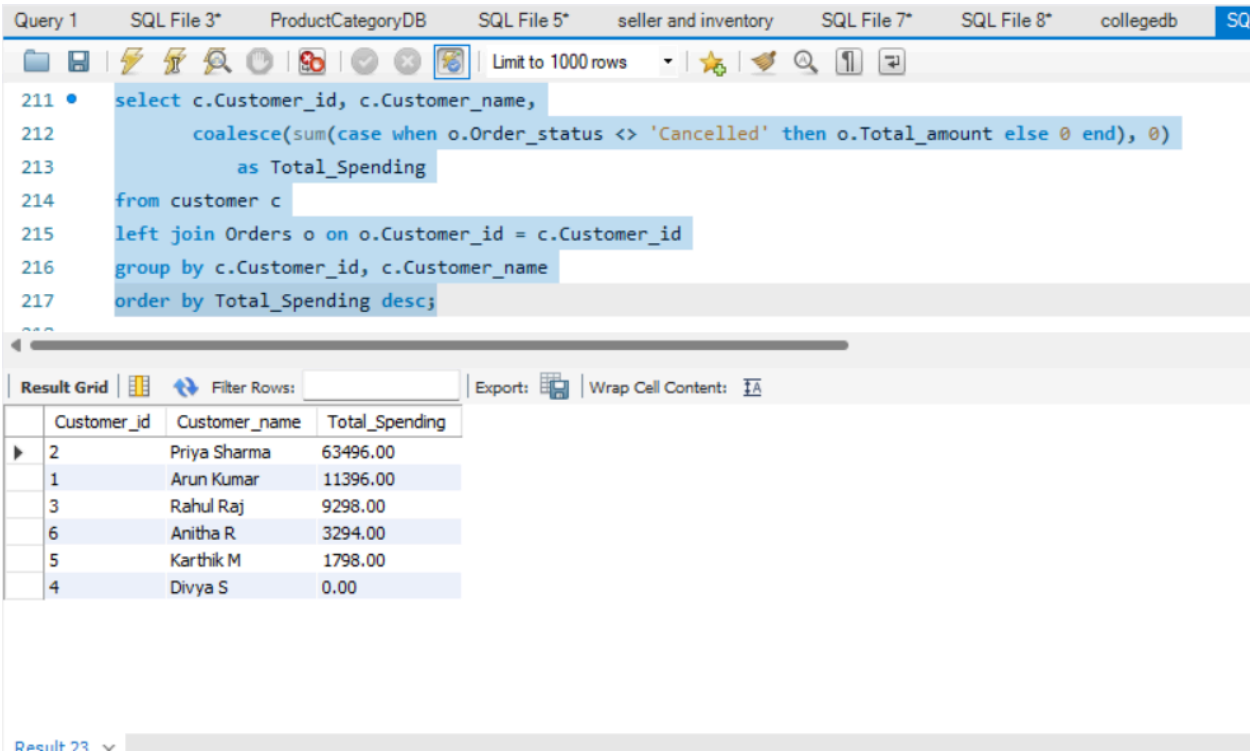
4) Total Quantity Sold



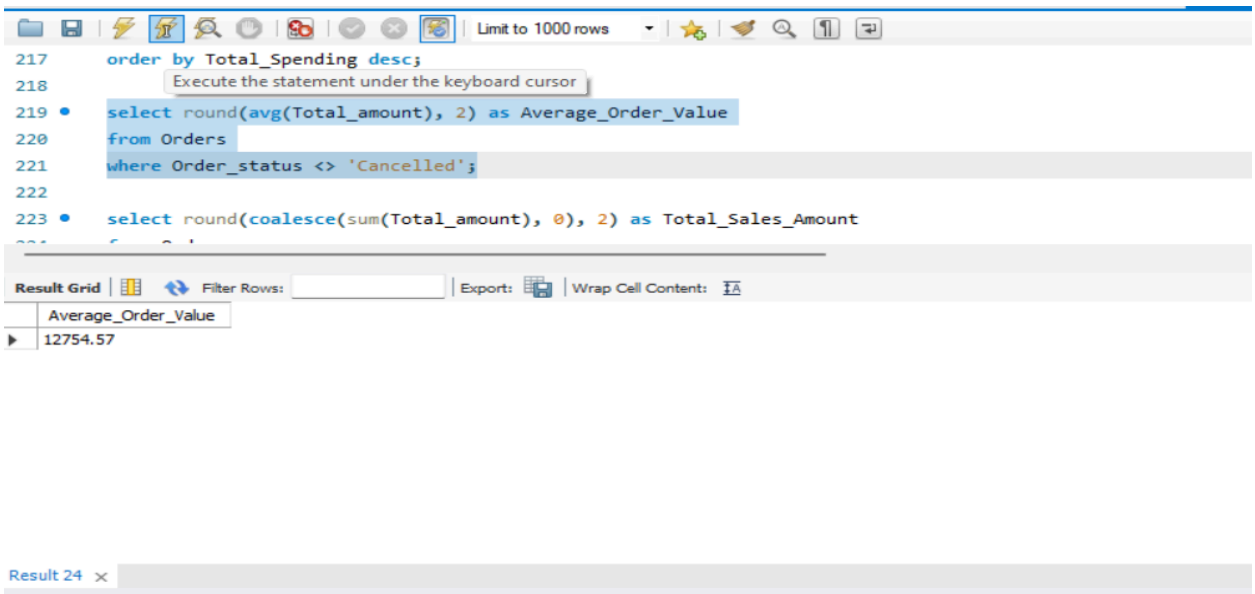
5) Maximum-Orders Customers



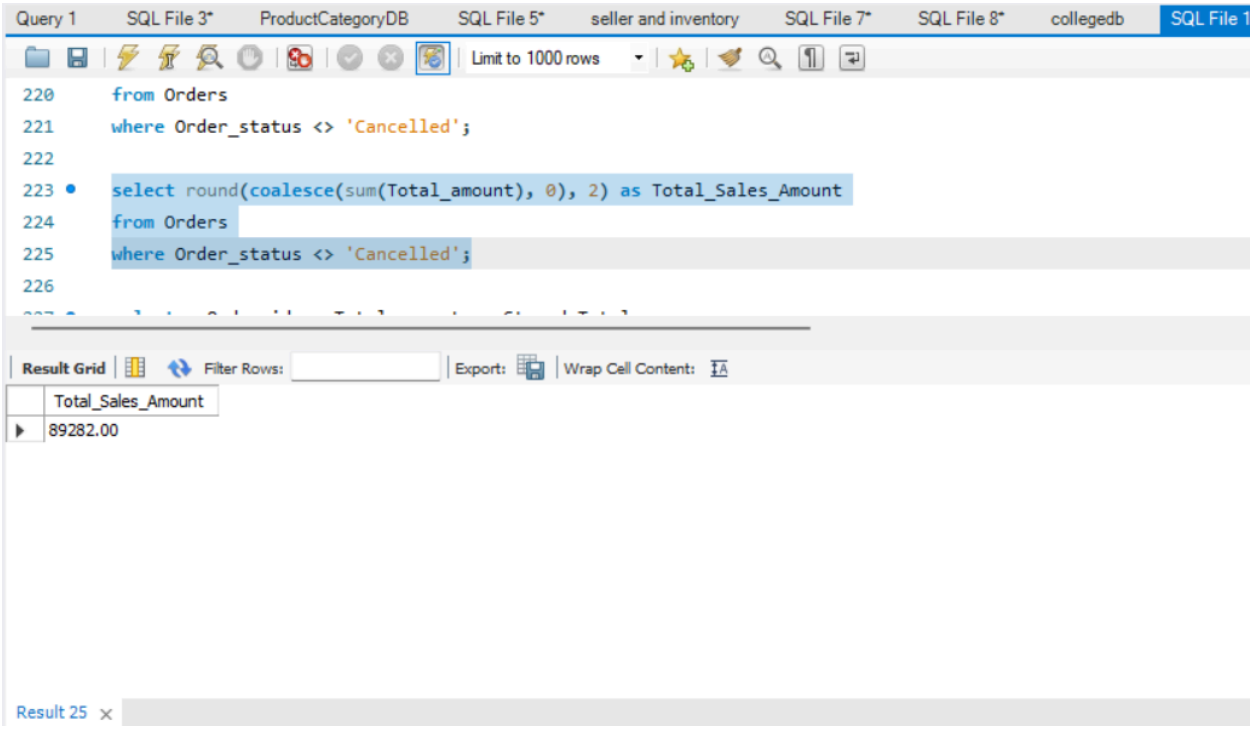
6) Customer Spending



7) Average Order Value



8) Total Sales Amount



10. Conclusion

The project demonstrates relational database design, referential integrity, data validation constraints, CRUD operations, joins, aggregation, and reporting in MySQL. The Order_Details table supports multiple products per order and provides the basis for purchase and sales analysis.